

Chapter 1: Data Mining

Foundations, Modeling Philosophies, and Key Tools

Based on: Mining of Massive Datasets (Leskovec, Rajaraman, Ullman)

Mahesh Shakya[1] & Siva Ram Dam

[1] NAAMII, Nepal & University of Tübingen, Germany

Gandaki University - Big Data Analytics

Masters in IT - Artificial Intelligence

Contents

Classic Data Mining vs. Machine Learning

Statistical Modeling: Distribution fitting

Computational Modeling: Summarization, Feature Extraction

Statistical Limits of Data Mining

Things to know: TF-IDF, Hash functions, Indexes, Base of Natural Log, Power Laws

1.1 What is Data Mining?

Data mining deals with:

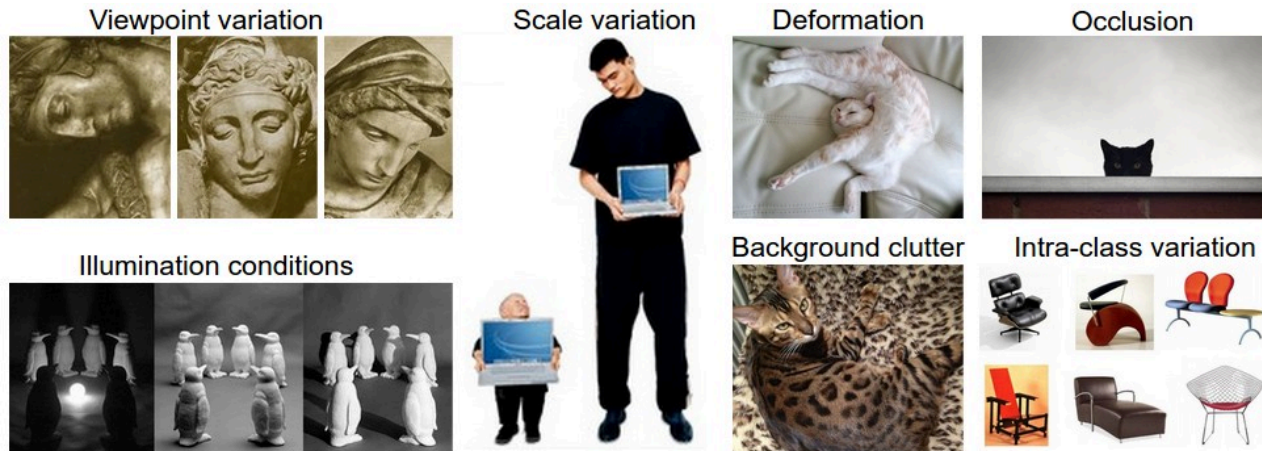
- **Powerful hardware** — enough compute/storage to touch huge datasets
- **Powerful programming systems** — MapReduce, streaming engines, distributed DBs
- **Efficient algorithms** — designed to scale

The goal: solve real problems in science, commerce, and beyond, at a scale where naive methods break down.

Two Ways to Think About "Modeling"

1. Data mining as Machine Learning

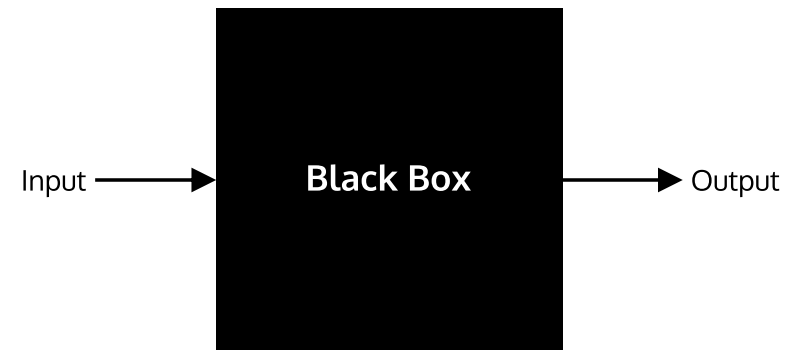
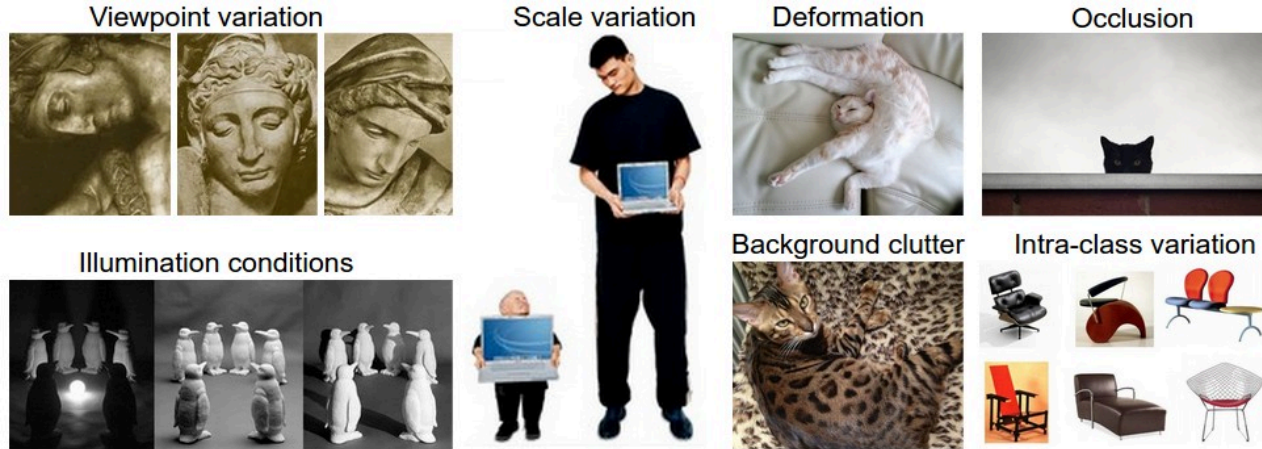
- Create a model *from* data
- The model is the product (a classifier, a predictor, a set of rules)



Machine Learning: When It's the Right Tool

Good fit when:

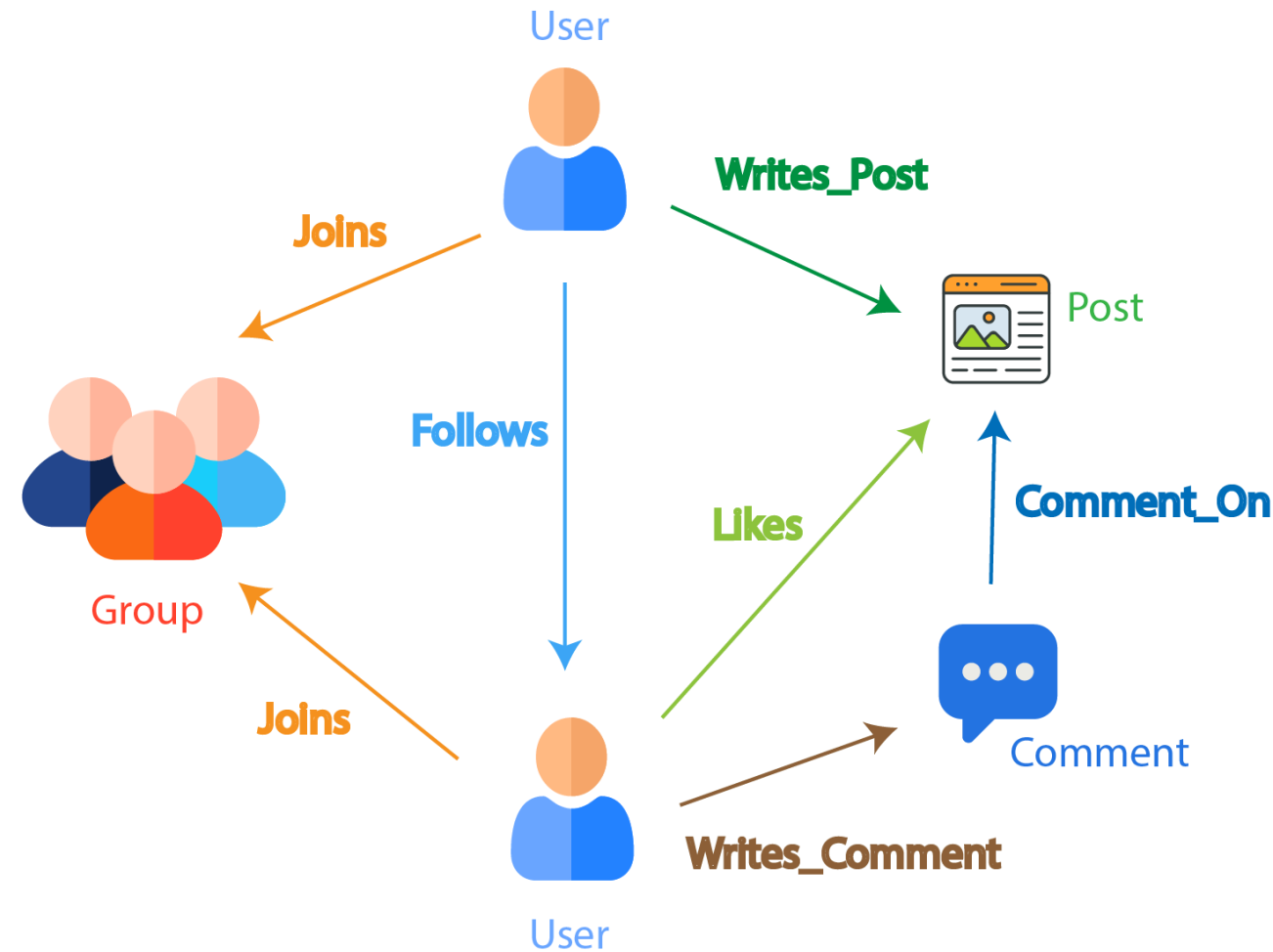
- We have little idea what the data says about the underlying problem
 - e.g., computer vision — no clean formula maps pixels → "cat"
- We don't need explainability, just accurate predictions



Machine Learning: When It's the Right Tool

Poor fit when:

- We can describe the mining goal directly (a clear query/rule suffices)
 - e.g., "find people's résumés on the web" → pattern/keyword search beats a trained classifier
- We need interpretable reasoning behind a decision (e.g., loan denial, medical diagnosis)



Two Ways to Think About "Modeling"

1. Data mining as Machine Learning

- Create a model *from* data
- The model is the product (a classifier, a predictor, a set of rules)

2. Data mining in General

- Build an *algorithm*
- e.g., Locality-Sensitive Hashing, stream-mining algorithms

We will take the Second one!

Two ways to think about "Modeling"

Statistical Modeling

Idea: Assume the data was drawn from some underlying distribution. Estimate that distribution.

- Example distribution family: **Gaussian (Normal)**
- Fitting it tells us the "shape" of the process generating the data, and lets us reason about new points, outliers, and confidence.

Goal: not just fit numbers, but **understand the problem better**.

Worked Example: Fitting a Gaussian

Suppose we observe 8 data points (e.g., response times in ms):

$$\{2, 4, 4, 4, 5, 5, 7, 9\}$$

Step 1 — Mean:

$$\mu = \frac{2 + 4 + 4 + 4 + 5 + 5 + 7 + 9}{8} = \frac{40}{8} = 5$$

Step 2 — Variance:

$$\sigma^2 = \frac{(2 - 5)^2 + (4 - 5)^2 \cdot 3 + (5 - 5)^2 \cdot 2 + (7 - 5)^2 + (9 - 5)^2}{8} = \frac{9 + 3 + 0 + 4 + 16}{8} = 4$$

Step 3 — Std. deviation: $\sigma = 2$

Interpreting the Fitted Model

We now have a model: $X \sim \mathcal{N}(\mu = 5, \sigma^2 = 4)$

This lets us answer questions the raw list can't easily answer:

- "How unusual is a new value of 13?" $\rightarrow$ that's $(13 - 5)/2 = 4$ standard deviations away $\rightarrow$ very unusual
- We can generate synthetic data, estimate probabilities of ranges, and detect outliers

This is the **statistical modeling** mindset: fit a distribution, then reason using its properties.

Computational Approaches to Modeling

Different from statistical modeling:

- We just want an **answer to a complex query about the data**
- We don't necessarily care about *how* the data was generated
- We accept **approximation** in exchange for speed/scale

Frequently Bought Together

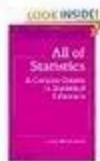


Price For All Three: \$258.02

Add all three to Cart

- ☒ **This item:** The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition (Springer Series in Statistics) by Trevor Hastie
- ☒ [Pattern Recognition and Machine Learning \(Information Science and Statistics\)](#) by Christopher M. Bishop
- ☒ [Pattern Classification \(2nd Edition\)](#) by Richard O. Duda

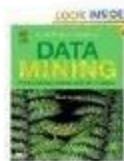
Customers Who Bought This Item Also Bought



[All of Statistics: A Concise Course in Statistical Inference](#) by Larry Wasserman



[Pattern Classification \(2nd Edition\)](#) by Richard O. Duda
★★★★☆ (27) \$117.25



[Data Mining: Practical Machine Learning Tools and Techniques](#) by Ian H. Witten
★★★★☆ (29) \$41.55



[Bayesian Data Analysis, Second Edition \(Texts in Probability and Statistics\)](#) by Andrew Gelman
★★★★☆ (10) \$56.20

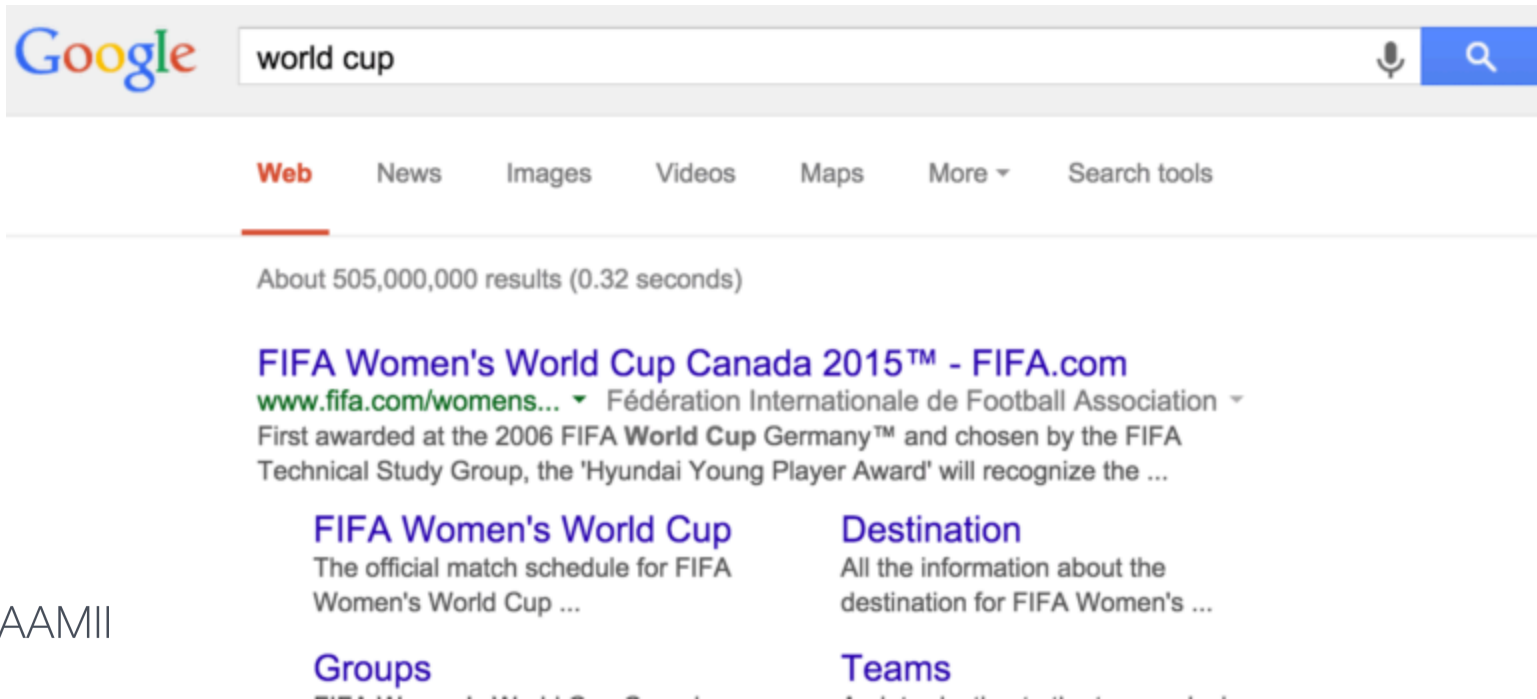


[Data Analysis Using Regression and Multilevel/Hierarchical Models](#) by Andrew Gelman
★★★★☆ (10) \$56.20

Computational Approaches to Modeling

Different from statistical modeling:

- We just want an **answer to a complex query about the data**
- We don't necessarily care about *how* the data was generated
- We accept **approximation** in exchange for speed/scale



Computational Approaches to Modeling

Different from statistical modeling:

- We just want an **answer to a complex query about the data**
- We don't necessarily care about *how* the data was generated
- We accept **approximation** in exchange for speed/scale

Two flavors:

1. **Summarization** — represent data compactly and approximately
2. **Feature Extraction** — keep the most prominent structure, discard the rest

Summarization Example: PageRank

Assign each web page a single number — its "importance."

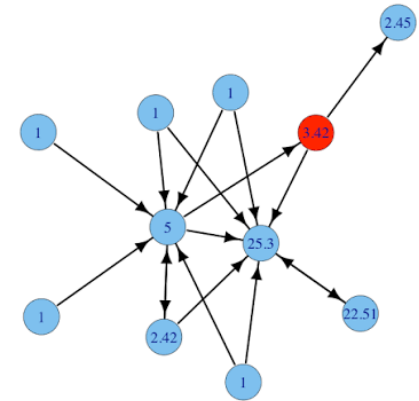
Intuition: PageRank = probability a random walker following links is at that page at any given time.

Tiny example — 3 pages, $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$ and $C \rightarrow B$:

Page	Out-links	Simplified rank intuition
A	→ B	Receives link only from C
B	→ C	Receives links from A and C
C	→ A, B	Receives link only from B

B has the most incoming "votes" → B ends up with the highest

PageRank.



Summarization Example: Clustering

Group points that are "close" into the same cluster.

Tiny example — 1-D points: $\{1, 2, 2, 8, 9, 10\}$

- Cluster 1: $\{1, 2, 2\} \rightarrow \text{centroid} \approx 1.67$
- Cluster 2: $\{8, 9, 10\} \rightarrow \text{centroid} = 9$

Instead of storing 6 numbers, we can summarize the dataset as **"2 groups centered at 1.67 and 9."**

Feature Extraction Example: Frequent Itemsets

Market-basket data — which items co-occur often?

Transaction	Items
T1	bread, milk
T2	bread, diapers, beer, eggs
T3	milk, diapers, beer, cola
T4	bread, milk, diapers, beer
T5	bread, milk, diapers, cola

- {bread, milk} appears in 3/5 transactions → **support = 0.6**
- {diapers, beer} appears in 3/5 transactions → **support = 0.6**



Feature Extraction Example: Similar Items

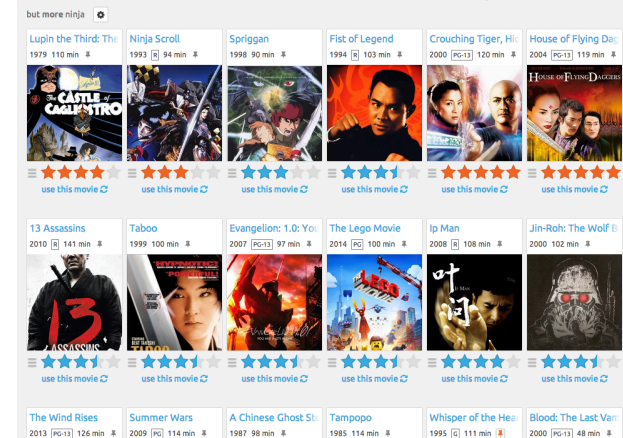
Collaborative filtering — find users (or items) that behave alike.

User	Movie A	Movie B	Movie C
Alice	5	4	1
Bob	5	5	2
Carol	1	2	5

- Alice and Bob rate similarly (high on A/B, low on C) → **similar users**
- Carol rates the opposite way → **dissimilar to Alice/Bob**

Recommend to Alice whatever Bob liked that she hasn't seen

Movies like **Castle in the Sky**



Contents

Classic Data Mining vs. Machine Learning

Statistical Modeling: Distribution fitting

Computational Modeling: Summarization, Feature Extraction

Statistical Limits of Data Mining

Things to know: TF-IDF, Hash functions, Indexes, Base of Natural Log, Power Laws

Statistical Limits: The Bonferroni Principle

Warning: if you test enough hypotheses on random data, some will look significant purely by chance.

Rule of thumb: before believing a "discovery," calculate the number of occurrences you'd *expect by chance alone*, assuming the data is random. If that expected count is already large, your finding is likely noise.

Worked Example: Bonferroni in Action

Setup (inspired by the classic "evil doers" scenario):

- 1,000 people, 100 hotels, tracked over 10 days
- $P(2 \text{ specific people at same hotel, same day}) = 1/100$
- Expected coincidences for **one pair** over 10 days: $10 \times \frac{1}{100} = 0.1$
- Using Poisson approx ($\lambda = 0.1$), $P(\geq 2 \text{ coincidences for that pair}) \approx 0.0047$

Now multiply by all pairs: $\binom{1000}{2} = 499,500$

Expected "suspicious" pairs $\approx 499,500 \times 0.0047 \approx 2,350$

~2,350 innocent pairs will look suspicious purely by chance — even though each individual event was rare.

Things useful to know

TF-IDF

Hash Functions

Secondary Storage

base of natural log e

Power Laws

TF-IDF: Term Frequency–Inverse Document Frequency

- **TF** (f_{ij}): how often term i appears in document j , normalized by the max term frequency in that document
- **IDF**: $IDF_i = \log_2(N/n_i)$, where n_i = # documents containing term i , N = total documents

Rare terms that appear a lot *in one document* score highest — they're the most distinguishing.

Worked Example: TF-IDF

3 short documents:

- D1: "data mining data science"
- D2: "data science rocks"
- D3: "mining gold"

For term "**data**": max raw count in D1 is 2 (for "data") $\rightarrow TF_{\text{data}, D1} = 2/2 = 1.0$

$N = 3$ documents; "data" appears in $n_i = 2$ documents (D1, D2):

$$IDF_{\text{data}} = \log_2(3/2) \approx 0.585$$

$$TF\text{-}IDF_{\text{data}, D1} = 1.0 \times 0.585 = 0.585$$

Compare to "**gold**": appears in only 1 doc $\rightarrow IDF = \log_2(3/1) = 1.585$ (much higher

→ "gold" is more distinctive than "data").

Hash Functions

A hash function maps keys to a small range of bucket numbers — essential for indexing, LSH, and stream algorithms.

Simple example: $h(k) = k \bmod 10$

Key	Bucket
23	3
47	7
103	3
58	8

Keys 23 and 103 **collide** in bucket 3 — a real hash function needs collision handling (chaining, probing, etc.).

Indexes

A data structure enabling **fast retrieval by value**, without scanning the whole dataset.

- Example: an index on "customer_id" lets you jump straight to that customer's records
- A **hash table** is one simple way to build an index — hash the key, land directly on the bucket holding matching records

Without an index: scan N records. With a hash index: $\sim O(1)$ lookup.

INDEX

ABC, 164, 321_n
academic journals, 262, 280–82
Adobe eBook Reader, 148–53
advertising, 36, 45–46, 127, 145–46, 167–68, 321_n
Africa, medications for HIV patients in, 257–61
Agee, Michael, 223–24, 225
agricultural patents, 313_n
Aibo robotic dog, 153–55, 156, 157, 160
AIDS medications, 257–60
air traffic, land ownership vs., 1–3
Akerlof, George, 232
Alben, Alex, 100–104, 105, 198–99, 295, 317_n
alcohol prohibition, 200
Alice's Adventures in Wonderland (Carroll), 152–53
Anello, Douglas, 60
animated cartoons, 21–24
antiretroviral drugs, 257–61
Apple Corporation, 203, 264, 302
architecture, constraint effected through, 122, 123, 124, 318_n
archive.org, 112
 see also Internet Archive
archives, digital, 108–15, 173, 222, 226–27
Aristotle, 150
Armstrong, Edwin Howard, 3–6, 184, 196
Arrow, Kenneth, 232
art, underground, 186
artists:
 publicity rights on images of, 317_n
 recording industry payments to, 52, 58–59, 74, 195, 196–97, 199, 301, 329_n–30_n

Secondary Storage

- Disks are physically organized into **blocks**, and blocks into **cylinders**
- Reading a whole block is roughly as cheap as reading one byte from it — algorithms should **minimize the number of blocks touched**, not the number of bytes
- This is *why* many data-mining algorithms are designed around sequential, block-friendly access patterns rather than random access

The Base of the Natural Log, e

$$e = \lim_{x \rightarrow \infty} \left(1 + \frac{1}{x}\right)^x$$

x	$(1 + 1/x)^x$
10	2.5937
100	2.7048
1,000	2.7169
10,000	2.7181
$\rightarrow \infty$	2.71828...

Useful approximation: for small a , $(1 + a)^b \approx e^{ab}$

Example: $(1.01)^{50} \approx e^{0.01 \times 50} = e^{0.5} \approx 1.6487$ (actual value: 1.6446 — close!)

Power Laws

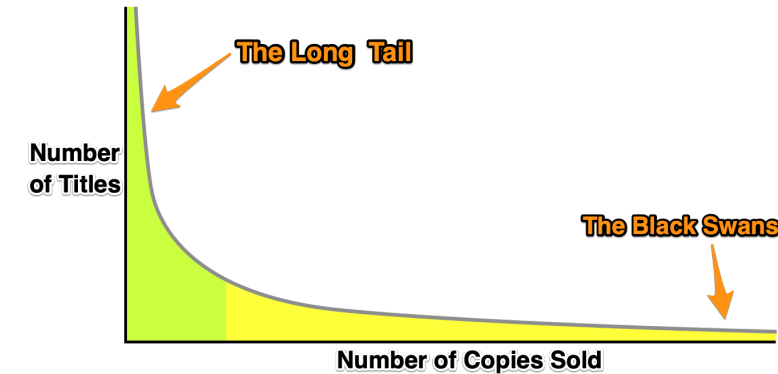
Many phenomena relate two variables by:

$$\log y = b + a \log x \quad \Longleftrightarrow \quad y = c \cdot x^a$$

A straight line **on a log-log plot** — the hallmark of a power law / "long tail."

Examples:

- **Web graph node degree** — most pages have very few links in; a handful of hubs have enormous numbers
- **Product sales** — most products sell rarely; a few "blockbusters" dominate sales
- **Zipf's Law** — word frequency in text

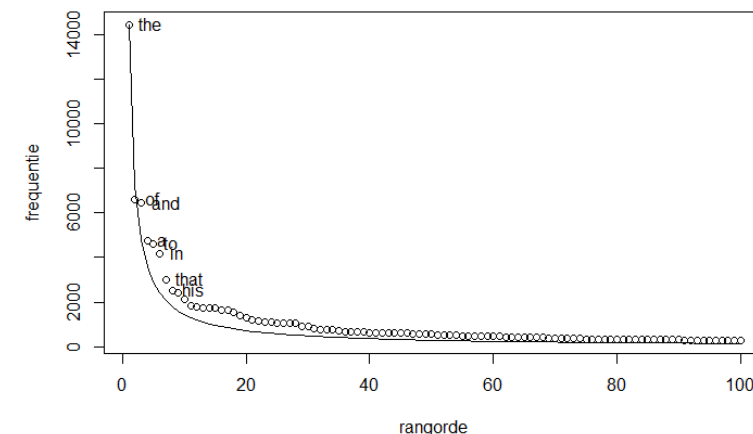


Worked Example: Zipf's Law

Rank words by frequency in a large English corpus:

Rank (x)	Word	Relative frequency (y)
1	the	1.00
2	of	0.50
3	and	0.33
4	to	0.25
10	(10th word)	≈ 0.10

This follows $y \approx 1/x$ — i.e., $\log y = -\log x$ (slope $a = -1$). Plotting $\log(\text{rank})$ vs. $\log(\text{frequency})$ gives a near-straight line — the signature of a power law.



Recap

Concept	Core idea	Example used
Statistical modeling	Fit a distribution	Gaussian fit to 8 numbers
Machine learning	Learn from data when goals are fuzzy	Vision vs. résumé search
Summarization	Compress structure into a number/group	PageRank, clustering
Feature extraction	Keep the salient patterns	Frequent itemsets, collaborative filtering
Bonferroni Principle	Chance findings in big data	~2,350 false "coincidences"
TF-IDF	Weight distinguishing terms	"data" vs. "gold" scoring
Hash functions /		

Next Steps

Complete Exercises for Section 1.3

Questions?